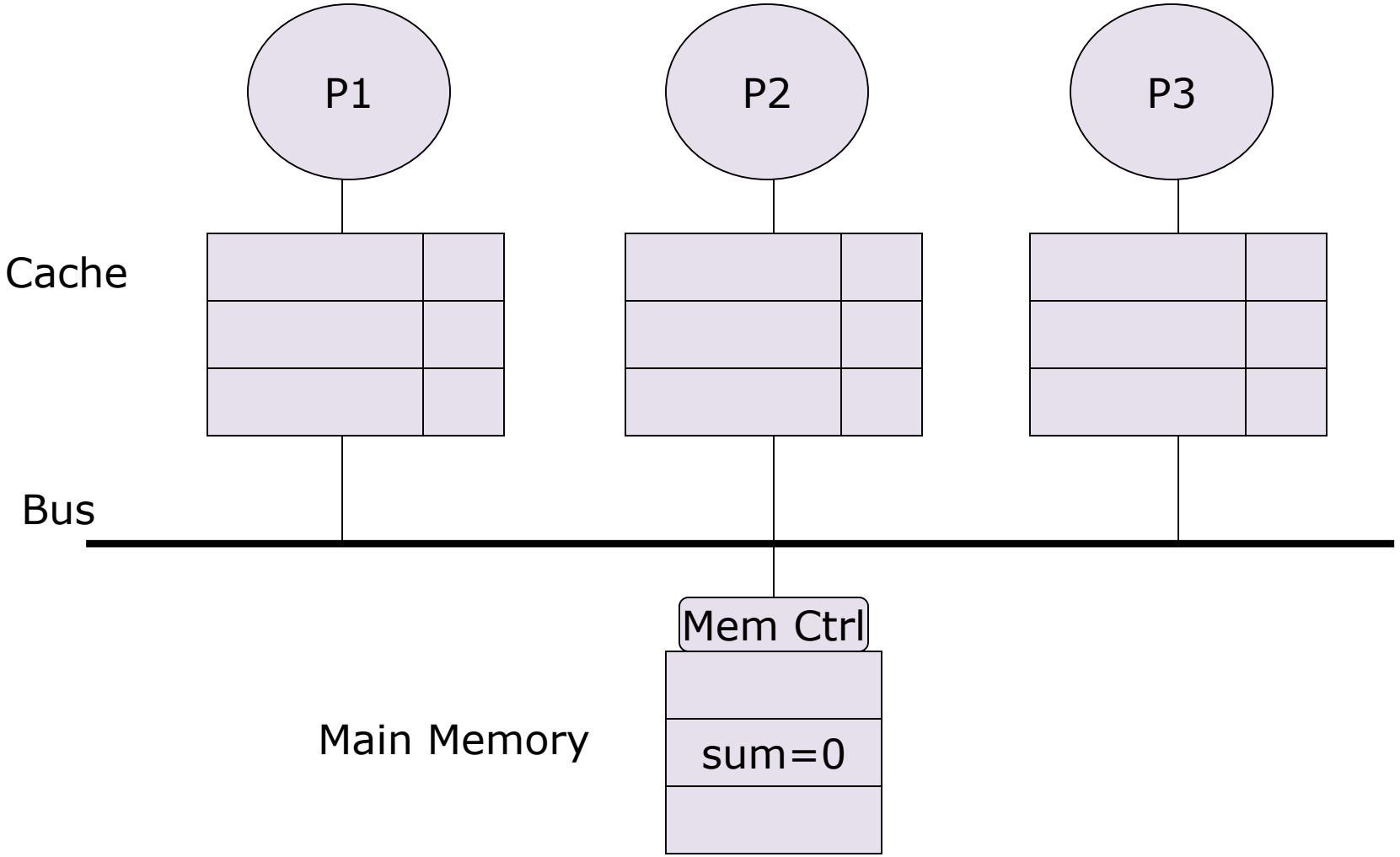
Questions

Q1: 什么是Cache coherence（缓存一致性）问题？可否举个例子。

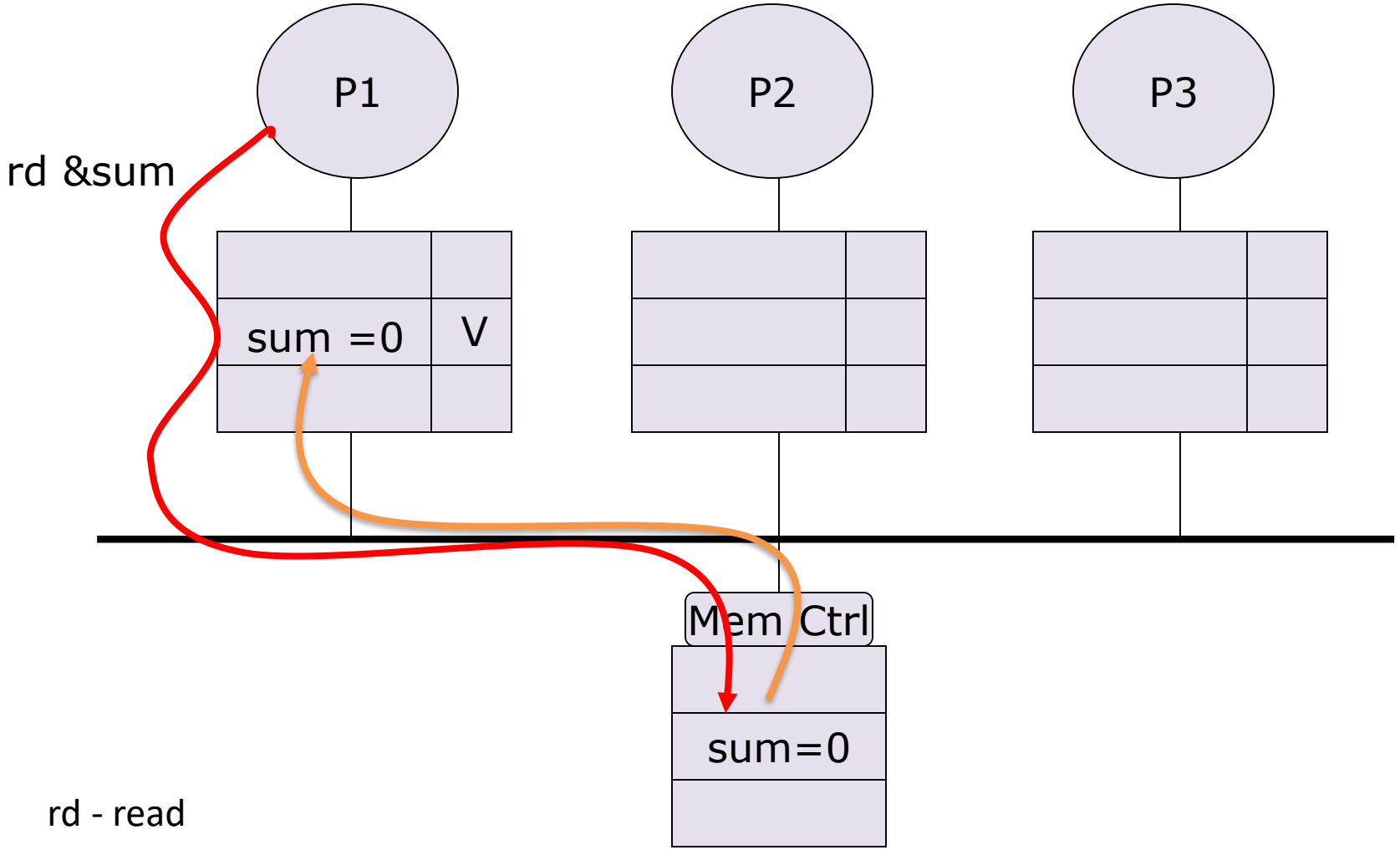
- **缓存一致性**是指在多处理器系统中，当多个处理器核心拥有自己的缓存时，如何确保它们共享数据的一致性。

- 当不同处理器缓存了同一内存位置的数据副本时，对这些副本的更新必须传播到所有其他缓存，以保持数据一致性。

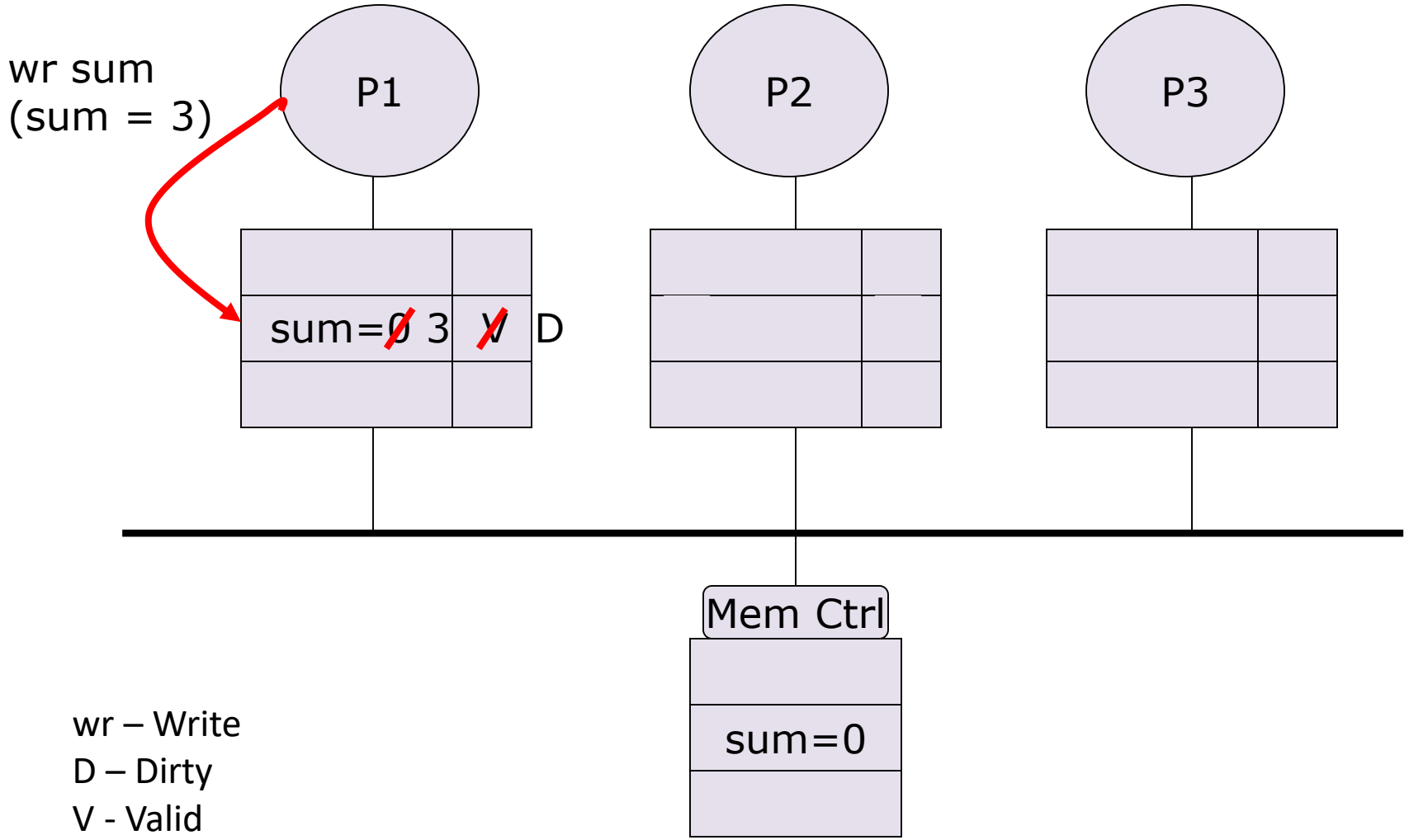
缓存一致性问题举例



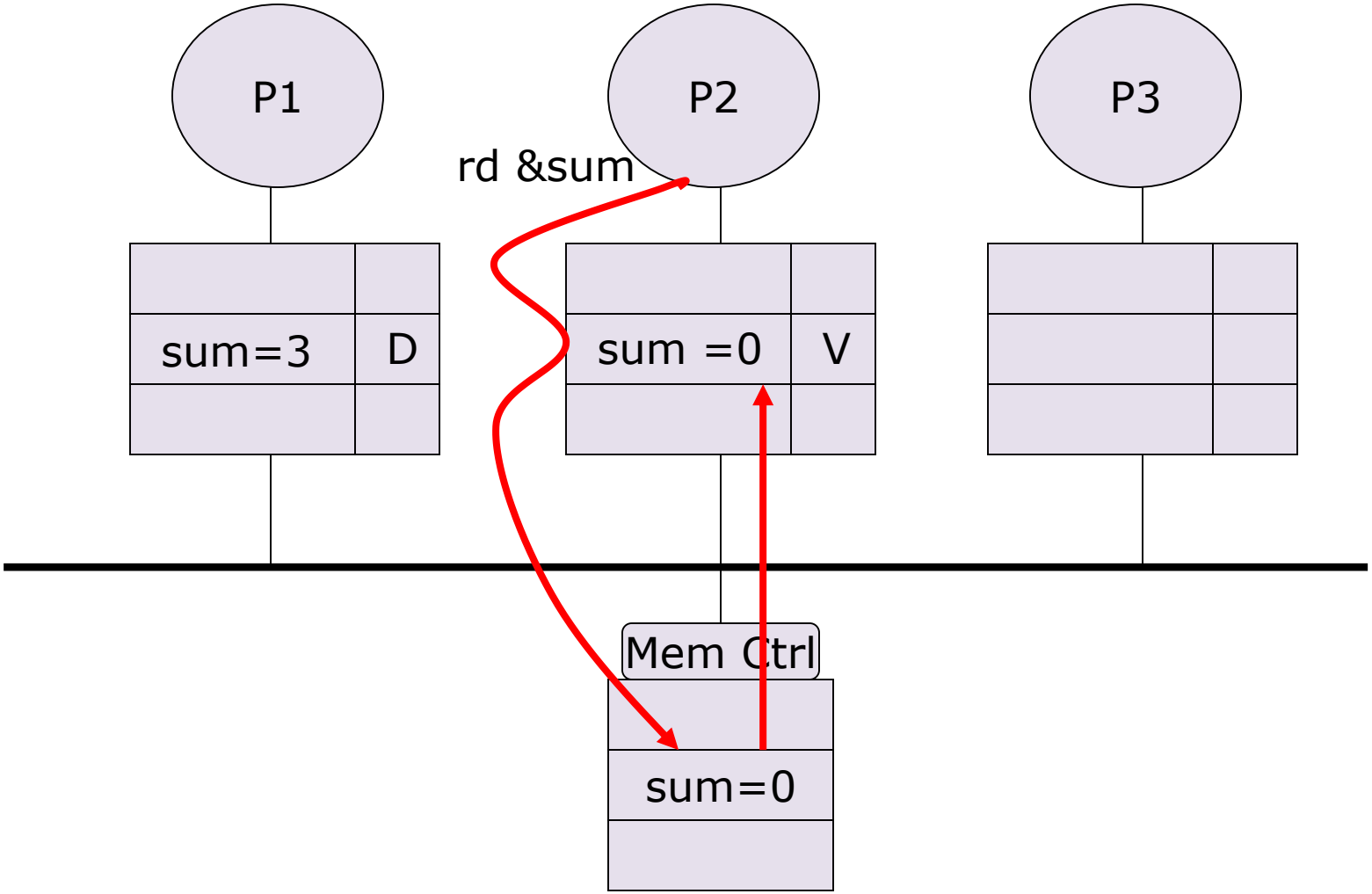
缓存一致性问题举例



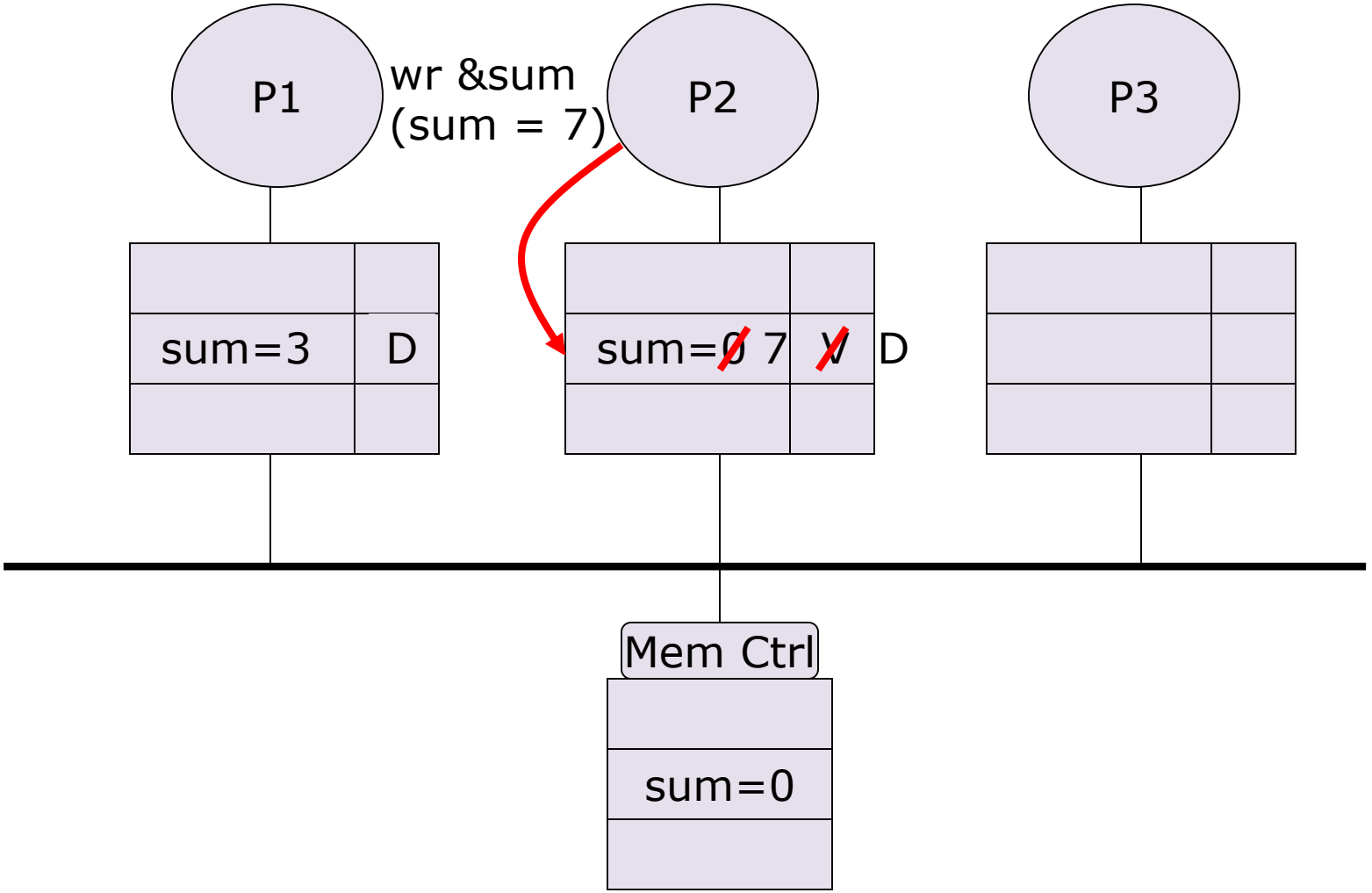
缓存一致性问题举例


wr sum
(sum = 3)
P1
P2
P3
sum=0 3 V D
Mem Ctrl
sum=0
wr – Write
D – Dirty
V - Valid

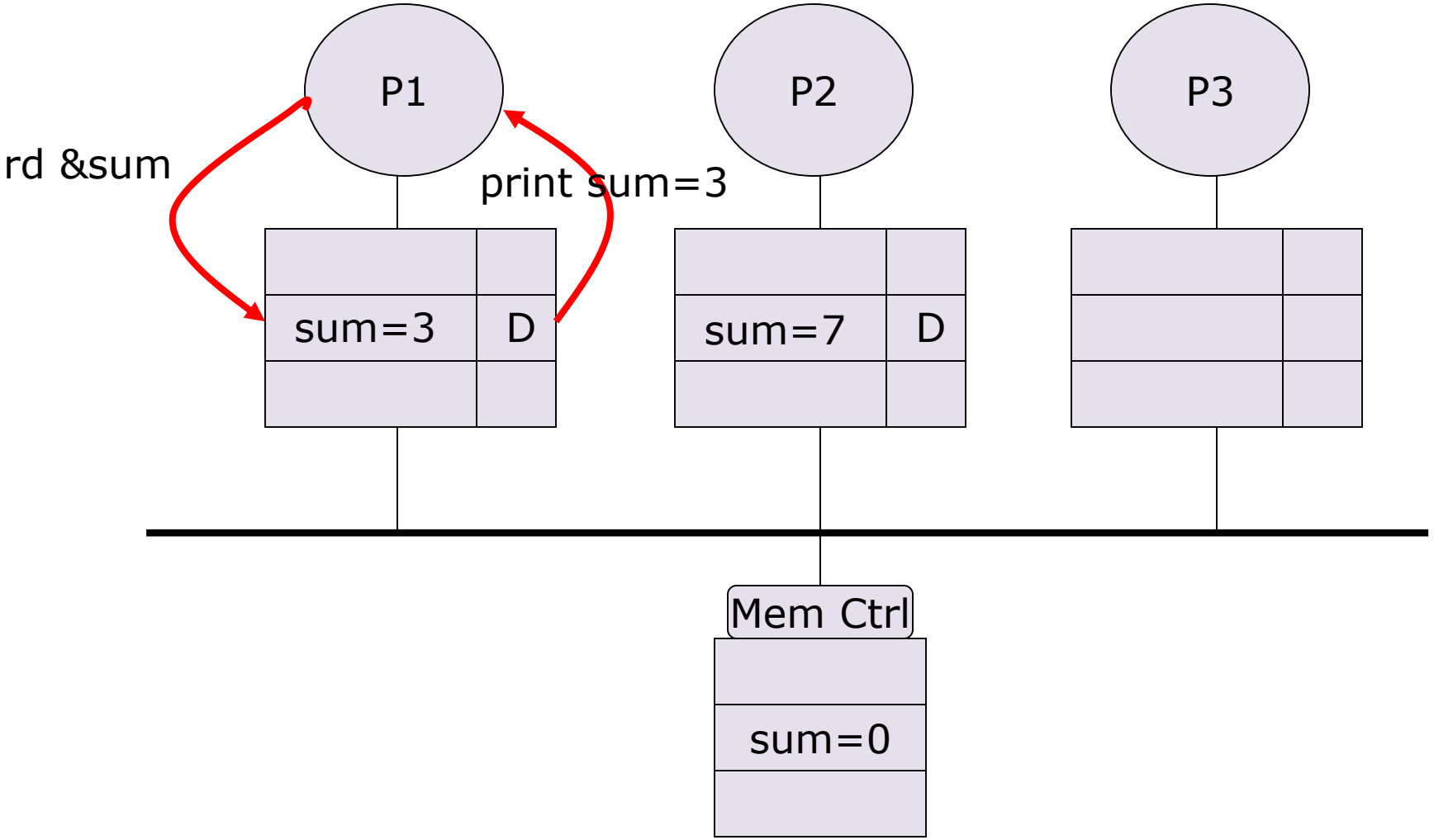
缓存一致性问题举例



缓存一致性问题举例



缓存一致性问题举例



Q2：什么是False Sharing （伪共享）问题？可否举个例子。

- **False Sharing（伪共享）发生在多个线程同时访问同一缓存行（Cache Line）中的不同变量时，导致不必要的缓存同步开销，从而降低程序性能。**

- **False Sharing 的原理：**
  - CPU 缓存以固定大小的块（通常 64 字节）加载数据，即使只修改其中一个变量，整个缓存行也会被标记为"脏"（Dirty），并触发缓存一致性协议的同步操作。
  - **伪共享：**如果两个线程分别修改同一个缓存行中的不同变量，即使它们没有真正的数据依赖关系，CPU 仍然会强制同步缓存，导致性能下降。

Q2：什么是False Sharing （伪共享）问题？可否举个例子。

```
struct Data {
    int x;  // Thread A 频繁修改 x
    int y;  // Thread B 频繁修改 y
};
Data data;  // x 和 y 在同一个缓存行
```

- Thread A 不断修改 data.x，导致 CPU 缓存行被标记为 修改 (Modified)
- Thread B 修改 data.y 时，由于 y 和 x 在同一个缓存行，CPU 必须：
  1. 使 Thread A 的缓存行失效 (Invalidate)
  2. 从内存重新加载缓存行 (Read)
  3. 修改 y 并标记为"修改" (Modified)
- 这样，即使 x 和 y 没有数据依赖关系，缓存行仍然不断在 CPU 核心之间同步，导致性能下降。

Q3: 什么是Thread Safety（线程安全）问题？

如何解决线程安全性问题？

- **线程安全（Thread Safety） 指在多线程环境下，程序或数据结构能够正确、一致地工作，而不会因并发访问导致数据损坏、逻辑错误或不可预测的行为。**

- **解决方法：**
  - **选用线程安全的函数**
  - **互斥锁：确保同一时间只有一个线程能访问共享资源**
  - **原子操作：确保操作不可分割**
  - **局部存储：让每个线程拥有独立的变量副本，避免共享数据**
  - **……**

Q4: OpenMP与pthreads都是多线程编程，它们有何区别?

特性	OpenMP	pthreads
线程创建	自动(由编译器/运行时处理)	手动(pthread_create)
线程同步	内置同步结构	需显式使用互斥锁/条件变量
负载均衡	自动调度	手动实现
可移植性	跨平台(需编译器支持)	UNIX-like系统
学习曲线	较简单	较复杂
控制粒度	较粗	精细
调试难度	较容易	较困难